



Diseño e Implementación de un Analizador

Sintáctico para Rondas de Truco

Entrega Final

Andres Leon Barberan; Geremias Benjamin Castillo; Ignacio Agustín Esquenón; Sebastian Exequiel Gomez;
Juan Cruz Senicen Acosta

Teoría de la Computación

Resumen

Se presenta el diseño e implementación de un analizador léxico-sintáctico completo para un lenguaje formal de dominio específico orientado a la descripción de partidas del juego de cartas Truco. El sistema recibe una cadena de texto que representa la secuencia de cantos y respuestas de una o más rondas, y determina si dicha secuencia es sintácticamente válida de acuerdo con una gramática libre de contexto no ambigua definida en notación BNF. La gramática, verificada formalmente como de clase LL(1) mediante el cálculo de los conjuntos FIRST y FOLLOW, modela la estructura jerárquica de los desafíos de envío y truco junto con sus posibles escaladas, y emplea recursión para representar la repetición de rondas dentro de una partida. El analizador léxico se implementó con el módulo de expresiones regulares de Python, produciendo una secuencia de tokens que alimenta al parser. El analizador sintáctico se construyó según la técnica de descenso recursivo e incorpora un mecanismo de recuperación de errores en modo pánico, que reporta la línea del error y permite continuar el análisis. Como salida, el sistema genera un árbol sintáctico abstracto visualizado con formato de árbol. Se diseñó un conjunto de casos de prueba representativos, tanto válidos como inválidos, cuyos resultados confirman que el analizador acepta correctamente las cadenas que se ajustan a la gramática y rechaza aquellas que presentan errores léxicos o sintácticos.

1. Introducción

El Truco, juego de naipes propio de la cultura rioplatense, posee un sistema de reglas que rige no sólo el resultado de las manos, sino también la estructura de la comunicación entre jugadores. Los cantos *-envío, truco, retruco y vale cuatro-* y sus respectivas respuestas conforman una secuencia altamente pausada, cuya validez depende de restricciones contextuales y de jerarquía. Esta regularidad sintáctica convierte al Truco en un dominio propicio para ser modelado mediante un lenguaje formal.

El presente trabajo aborda el diseño e implementación de un analizador léxico-sintáctico completo para un lenguaje de descripción de partidas de Truco. Se busca proporcionar una herramienta que, a partir de una cadena de texto que represente una secuencia de juego, determine automáticamente si dicha secuencia es estructuralmente válida y, en caso contrario, informe la ubicación y la naturaleza de los errores encontrados.

Los objetivos específicos del proyecto son: definir formalmente una gramática libre de contexto no ambigua empleando notación BNF; construir un analizador léxico basado en expresiones regulares; implementar un parser descendente recursivo con un mecanismo de recuperación de errores en modo pánico; generar un árbol sintáctico abstracto como representación de la entrada analizada; y validar el sistema mediante casos de prueba representativos.

El equipo de trabajo estuvo conformado por cinco integrantes, con los siguientes roles: Ignacio Esquenón como líder de proyecto, responsable de la coordinación y el control de versiones; Andrés León Barberan como arquitecto de gramática, encargado del diseño formal de la GLC; Geremias Castillo como desarrollador del analizador léxico; Sebastián Gómez como desarrollador del parser descendente recursivo; y Juan Cruz Senicen Acosta como documentador y tester, a cargo del informe técnico y de los casos de prueba. Para la gestión colaborativa, se empleó un repositorio Git alojado en GitHub, donde se registró el historial de contribuciones.

2. Desarrollo

El desarrollo del analizador se sustenta en un conjunto de conceptos formales de la teoría de la computación y del diseño de compiladores, los cuales se detallan a continuación.

2.1 Marco Teórico

Gramáticas formales. Una gramática formal es una 4-tupla compuesta por un conjunto finito de variables, un conjunto finito de terminales, un conjunto finito de reglas de producción y un símbolo inicial. La jerarquía

de Chomsky [1] clasifica estas gramáticas según su poder generativo. En el presente trabajo se utiliza una **Gramática Libre de Contexto (GLC)**, cuyas producciones poseen una única variable en el lado izquierdo y permiten modelar estructuras anidadas y recursivas presentes en los lenguajes de programación y, como se demuestra aquí, en los cantos del Truco.

Notación BNF/EBNF. Para especificar la gramática se emplea la notación BNF (Backus-Naur Form). Sus extensiones EBNF, que introducen operadores como la barra vertical para alternativas y los paréntesis para agrupamiento, son utilizadas por Nystrom [2] para describir la sintaxis del lenguaje Lox y facilita una traducción directa a un parser recursivo.

Autómatas y análisis léxico. El análisis léxico se basa en la teoría de **autómatas finitos (DFA/NFA)** y **expresiones regulares** [1]. Dado que la especificación de los tokens de un lenguaje constituye un lenguaje regular, es posible construir un DFA que reconozca cada lexema. La implementación del lexer utilizando el módulo “re” de Python sigue el enfoque mencionado por Nystrom [2], donde una tabla de patrones regex reemplaza la codificación manual de un autómata finito.

Autómatas y análisis sintáctico. El análisis sintáctico de una GLC requiere un **autómata de pila (PDA)** [1], que añade una memoria de pila a la capacidad de cómputo de un autómata finito. La técnica de **parsing descendente recursivo (Recursive Descent)** [2] implementa este autómata de forma natural: la pila de llamadas del programa hace de pila del autómata. Cada no terminal de la gramática se traduce en una función que consume tokens, y las producciones alternativas se deciden mediante un único token de anticipación.

Gramáticas LL(1) y conjuntos FIRST/FOLLOW. Para que un parser descendente recursivo pueda decidir sin retroceso, la gramática debe pertenecer a la clase **LL(1)** [3]. Esto exige que las producciones alternativas de un mismo no terminal posean conjuntos **FIRST** disjuntos y que, en caso de producciones anulables, la intersección entre **FIRST** y **FOLLOW** sea vacía. La verificación formal de estas condiciones se realizó siguiendo las definiciones de Aho et al. [3], mientras que los métodos `check()` y `match()` del parser implementan la decisión basada en el lookahead [2].

Árbol Sintáctico Abstracto (AST). Como representación intermedia, el parser genera un **Árbol Sintáctico Abstracto** [2]. A diferencia del árbol de

parseo concreto, el AST omite los nodos puramente sintácticos que no aportan significado, facilitando un hipotético procesamiento posterior.

Manejo de errores (modo pánico). Para cumplir con el requisito de reporte y recuperación de errores, se implementó la estrategia de **panic mode** [2], [3]. Ante un error sintáctico, el parser descarta tokens de entrada hasta encontrar un punto de sincronización (en este proyecto, el separador de rondas “;” o el fin de archivo), lo que permite reanudar el análisis y reportar múltiples errores en una sola ejecución.

2.2 Diseño de la Gramática

Se definió una gramática libre de contexto (GLC) para modelar el lenguaje de descripción de partidas de Truco. Formalmente, la gramática es la 4-tupla, donde:

$S = \text{PARTIDA}$

$V = \{\text{PARTIDA, RESTO_PARTIDA, RONDA, ENVIDO, R1, R2, R3, RE, TRUCO, RT, RT1}\}$

$\Sigma = \{\text{envido, real_envido, falta_envido, truco, retruco, vale_cuatro, quiero, no_quiero, ;}\}$

El conjunto R se presenta en la siguiente tabla:

Tabla 1. Reglas de Producción en BNF.

No Terminal	Producción
PARTIDA	→ RONDA RESTO PARTIDA
RESTO_PARTIDA	→ “;” RONDA RESTO_PARTIDA ϵ
RONDA	→ ENVIDO TRUCO TRUCO
ENVIDO	→ “envido” R1 “real_envido” R3 “falta_envido” RE
R1	→ “envido” R2 “real_envido” R3 “falta_envido” RE RE ;
R2	→ “real_envido” R3 “falta_envido” RE RE ;
R3	→ “falta_envido” RE RE ;
RE	→ “quiero” “no_quiero” ;
TRUCO	→ “truco” RT ϵ
RT	→ “retruco” RT1 RE ;
RT1	→ “vale_cuatro” RE RE ;

La gramática del lenguaje de partidas de Truco se diseñó para evaluar exclusivamente la validez estructural de las secuencias de cantos, abstrayéndose de quién gana o qué puntaje se acumula. La hipótesis de trabajo es que una partida se reduce a una sucesión de rondas, cada una de las cuales puede contener una fase de envido y una fase de truco, ambas opcionales. Si en



una ronda no se canta nada, simplemente se la representa sin tokens; el separador "," entre rondas vacías produce cadenas como “,” o “,,”, sintácticamente correctas, que corresponden a manos jugadas sin incidencias verbales. Se volverá sobre este punto en la discusión de las decisiones de diseño.

El conjunto R se presenta en la Tabla 1 en notación BNF.

Con el propósito de verificar formalmente que la gramática pertenece a la clase LL(1) y, por tanto, puede ser analizada mediante un parser descendente recursivo sin retroceso, se calcularon los conjuntos FIRST y FOLLOW. Los resultados se muestran en la Tabla 2.

Tabla 2. Conjuntos FIRST y FOLLOW.

No Terminal	FIRST	FOLLOW
PARTIDA	{envido, real_envido, falta_envido, truco, ;, ε}	{S}
RESTO_PARTIDA	{;, ε}	{S}
RONDA	{envido, real_envido, falta_envido, truco, ε}	{;, S}
ENVIDO	{envido, real_envido, falta_envido}	{truco, ;, S}
R1	{envido, real_envido, falta_envido, quiero, no_quiero}	{truco, ;, S}
R2	{real_envido, falta_envido, quiero, no_quiero}	{truco, ;, S}
R3	{falta_envido, quiero, no_quiero}	{truco, ;, S}
RE	{quiero, no_quiero}	{truco, ;, S}
TRUCO	{truco, ε}	{;, S}
RT	{retruco, quiero, no_quiero}	{;, S}
RT1	{vale_cuatro, quiero, no_quiero}	{;, S}

A partir de estos conjuntos, se verificaron las tres condiciones, caracterizan a una gramática LL(1): (1) los FIRST de las producciones alternativas de un mismo no terminal deben ser disjuntos; (2) a lo sumo una de ellas puede derivar la cadena vacía ε; y (3) si una producción deriva ε, el FIRST de cualquier otra alternativa debe ser disjunto con el FOLLOW del no terminal [3].

La Tabla 3 resume esta verificación (el símbolo “\$” corresponde a “EOF”).

Tabla 3. Verificación de las condiciones LL(1).

No Terminal	¿Los FIRST de cada par de producciones son disjuntos?	¿Solo una producción deriva ε?	Si una producción deriva ε, ¿los FIRST de las otras producciones son disjuntos con FOLLOW?
PARTIDA	Solo tiene una producción, no aplica	-	-
RESTO_PARTIDA	{;} y {ε} disjuntos	Solo ε	{;} ∩ {S} = ∅
RONDA	{envido, real_envido, falta_envido} y {truco, ε} disjuntos	Solo TRUCO puede derivar ε	{envido, real_envido, falta_envido} ∩ {;, S} = ∅
ENVIDO	{envido}, {real_envido}, {falta_envido} disjuntos	Ninguna	-
R1	{envido}, {real_envido}, {falta_envido}, {quiero, no_quiero} disjuntos	Ninguna	-
R2	{real_envido}, {falta_envido}, {quiero, no_quiero} disjuntos	Ninguna	-
R3	{falta_envido}, {quiero, no_quiero} disjuntos	Ninguna	-
RE	{quiero}, {no_quiero} disjuntos	Ninguna	-
TRUCO	{truco}, {ε} disjuntos	Solo ε	{truco} ∩ {;, S} = ∅
RT	{retruco}, {quiero, no_quiero} disjuntos	Ninguna	-
RT1	{vale_cuatro}, {quiero, no_quiero} disjuntos	Ninguna	-

Como se observa en la Tabla 3, todos los no terminales satisfacen las condiciones requeridas. En los casos de RESTO_PARTIDA, RONDA y TRUCO, donde una de las alternativas deriva la cadena vacía, la intersección entre el FIRST de la otra alternativa y el FOLLOW correspondiente resulta vacía. La gramática del lenguaje de partidas de Truco es **LL(1) y no ambigua**.



2.3 Implementación

El sistema fue desarrollado en Python 3.13, utilizando exclusivamente la biblioteca estándar. La elección del lenguaje respondió a su legibilidad, su amplia adopción en entornos académicos y la disponibilidad del módulo “re” [4] para el análisis léxico. La arquitectura sigue el modelo de dos fases presentado por Nystrom [2]: un analizador léxico que convierte la cadena de entrada en una secuencia de tokens, y un analizador sintáctico que procesa dichos tokens y construye el árbol sintáctico abstracto. Ambos componentes se integran en un programa principal que ofrece varios modos de ejecución.

El código fuente de la implementación final se encuentra en la carpeta `src/` del repositorio Git del proyecto.

2.3.1 Analizador Léxico

El analizador léxico se implementó en el módulo `lexer.py` mediante la clase `Scanner`. Su responsabilidad es recorrer la cadena de entrada carácter por carácter y agruparlos en tokens según una tabla de patrones definida mediante expresiones regulares.

La tabla de patrones asocia cada tipo de token con su expresión regular correspondiente. Los tokens definidos cubren exactamente el alfabeto Σ de la gramática:

ENVIDO, REAL_ENVIDO, FALTA_ENVIDO, TRUCO, RETRUCO, VALE_CUATRO, QUIERO, NO QUIERO y PUNTO_Y_COMA (para el separador de rondas “;”).

El bucle principal, implementado en el método `tokenize()`, itera sobre la cadena de entrada. En cada posición, prueba los patrones en orden hasta encontrar una coincidencia, consume el lexema y, según el tipo de token, actúa en consecuencia: los patrones correspondientes a espacios, tabulaciones y comentarios (líneas que comienzan con #) se descartan sin generar token; los saltos de línea incrementan un contador interno para el reporte de errores; cualquier otro patrón produce un token válido que se añade a la lista de salida. Los caracteres no reconocidos por ningún patrón se agrupan bajo un token especial de tipo `ERROR`, lo que permite reportar errores léxicos sin interrumpir el análisis. Al finalizar el recorrido, se añade un token `EOF` que marca el fin de la entrada.

2.3.2 Analizador Sintáctico

El parser se implementó en el módulo `parser.py` mediante la clase `Parser`. Se eligió la técnica de descenso recursivo por su adecuación a gramáticas LL(1) y por la traducción directa que permite entre las producciones gramaticales y el código.

La clase `Parser` recibe la lista de tokens generada por el lexer y mantiene un índice (`current`) sobre ella. Los métodos auxiliares `peek()`, `advance()`, `check()` y `match()`

constituyen la infraestructura básica para recorrer la secuencia de tokens. El método `check()` inspecciona el token actual sin consumirlo, mientras que `match()` lo consume si coincide con alguno de los tipos esperados. Ambos métodos integran un mecanismo de detección temprana de tokens de error léxico: si el token actual es de tipo `ERROR`, se reporta un mensaje específico y se descarta el token antes de continuar.

Cada no terminal de la gramática se implementó como un método de la clase. Así: `partida()`, `resto_partida()`, `ronda()`, `envido()`, `r1()`, `r2()`, `r3()`, `re()`, `truco()`, `rt()` y `rt1()`; recorren la entrada y construyen recursivamente un Árbol Sintáctico Abstracto (AST). Los nombres de los no terminales se escriben en mayúsculas en los nodos del AST para distinguirlos visualmente de los lexemas, que se representan en minúsculas.

El AST se construye mediante tuplas anidadas de Python. El primer elemento de cada tupla identifica el tipo de nodo (el nombre del no terminal), y los elementos restantes son sus hijos, que pueden ser nuevas tuplas, cadenas (lexemas) o `None` cuando un componente opcional está ausente. La visualización del AST se realiza mediante la función `print_ast()`, que recorre la estructura recursivamente y la imprime en consola con conectores de árbol (`|`, `├──`, `└──`), proporcionando una representación legible de la jerarquía sintáctica.

Para el manejo de errores sintácticos se implementó la estrategia de **modo pánico**. Cuando un método encuentra un token inesperado, el método `error()` registra un mensaje con el número de línea y el token encontrado, y a continuación descarta tokens de la entrada hasta alcanzar un punto de sincronización. Los sincronizadores elegidos fueron `PUNTO_Y_COMA` y `EOF`, dado que el primero separa rondas y permite reanudar el análisis en la ronda siguiente, mientras que el segundo indica el final de la entrada. El sincronizador no se consume durante el modo pánico, sino que se deja disponible para que el método que invocó a `error()` pueda procesarlo normalmente.

2.3.3 Programa Principal

La integración de ambos componentes se realiza en el módulo `main.py`. Este módulo ofrece tres modos de ejecución:

Modo interactivo:

Se invoca con “`python src/main.py`” y sin argumentos. El programa solicita la partida línea por línea; la entrada finaliza con una línea vacía. A continuación, se muestran los tokens generados, el árbol sintáctico abstracto y los errores detectados.

Modo de argumento directo:

Se pasa la cadena a analizar como argumento de la línea de comandos, por ejemplo “`python src/main.py "truco quiero ; envido no quiero"`”. El análisis se ejecuta de inmediato y el resultado se imprime en la consola.

Modo de archivo:

Mediante la opción `--file` se especifica un archivo de texto cuyo contenido completo será analizado. Ejemplo:

`"python src/main.py --file tests/casos_validos.txt"`.

Este modo es especialmente útil para ejecutar los conjuntos completos de pruebas provistas en el repositorio.

En todos los casos, el flujo de procesamiento es el mismo: la cadena de entrada se pasa al método `tokenize()` del Scanner, que devuelve una lista de tokens. Esta lista se suministra al constructor del Parser, y su método `parse()` retorna una tupla con el AST y la lista de errores. La función `analizar()` del programa principal se encarga de presentar los resultados en consola, mostrando tanto los tokens intermedios como el árbol sintáctico final y los mensajes de error, si los hubiera.

2.4 Casos de Prueba y Resultados

Se diseñó un conjunto de 55 casos de prueba (38 válidos y 17 inválidos) para verificar el comportamiento del analizador frente a entradas correctas, errores léxicos y errores sintácticos. El conjunto completo se encuentra en el archivo `tests/resultados_pruebas.md` del repositorio.

Tabla 4. Selección de casos de prueba representativos.

Entrada	Tipo	Salida Esperada	Estado
envido quiero	Válido	AST generado y mensaje de éxito	Éxito
envido no_quiero truco retruco no_quiero	Válido	AST generado con envido y truco anidados	Éxito
truco quiero ; envido no_quiero ; truco retruco vale_cuatro quiero	Válido	AST de partida con tres rondas y mensaje de éxito	Éxito
turco quiero	Inválido	Error léxico: token no reconocido 'turco'	Éxito
real_envido envido quiero	Inválido	Error sintáctico: se esperaba falta_envido o una respuesta (quiero / no_quiero)	Éxito
truco vale_cuatro quiero	Inválido	Error sintáctico: se esperaba retruco o una respuesta (quiero / no_quiero)	Éxito

En todos los casos, la salida obtenida coincidió con la esperada. El analizador aceptó correctamente las cadenas que se ajustan a la gramática y rechazó aquellas que presentan infracciones léxicas o sintácticas, reportando en cada caso la línea y una descripción del error encontrado. Los casos límite (rondas vacías, partidas de múltiples rondas y comentarios) fueron igualmente procesados según la especificación.

2.5 Discusión y Análisis

El proceso de diseño e implementación permitió contrastar los conceptos formales de la teoría de autómatas con los desafíos prácticos del desarrollo. Se analizan a continuación las dificultades, decisiones adoptadas y limitaciones del sistema

2.5.1 Dificultades Encontradas

La principal dificultad fue formalizar las reglas informales del Truco como una GLC no ambigua. La versión preliminar de la segunda entrega modelaba los resultados de cada mano y definía la ronda como una secuencia fija de tres manos sin recursividad. Este enfoque, que buscaba capturar todos los detalles del juego, resultó excesivamente rígido para representar una partida real y no cumplía el requisito de recursividad.

2.5.2 Decisiones de Diseño Adoptadas

Desarrollo iterativo.

Antes de abordar la implementación de la gramática completa, se construyó un prototipo a escala reducida (Micro-Truco) para que los desarrolladores se familiarizaran con la mecánica del parsing descendente y el lexer en un entorno controlado.

Eliminación de resultados explícitos.

La versión de la segunda entrega incluía *gana_nosotros*, *gana_ellos* y *mazo*. Como el analizador solo valida la estructura de los cantos, estos terminales fueron suprimidos por añadir complejidad innecesaria. La eliminación condujo a la posibilidad de rondas vacías: sin cantos ni resultado, una ronda que equivale a cero tokens, y secuencias como `; o ;;` son sintácticamente válidas (significa: Rondas sin cantos, solo se jugaron cartas).

Recursividad.

Para satisfacer el requisito del proyecto, la estructura fija de tres manos se reemplazó por:

`RESTO_PARTIDA → ";" RONDA RESTO_PARTIDA | ε.`

Esta producción recursiva por derecha, implementada en el método `resto_partida()`, permite una cantidad arbitraria de rondas separadas por `;"`.



Jerarquía fija del canto de Truco.

A diferencia del envideo, donde puede cantarse falta_envideo directamente, los desafíos de truco exigen el orden truco → retruco → vale_cuatro. Una gramática recursiva habría permitido secuencias sin límite ajenas al juego. Los no terminales RT y RT1 garantizan ese orden y evitan ambigüedades.

Unificación de respuestas.

Todas las respuestas se unificaron en el no terminal RE, simplificando la gramática y alineándola con el método `re()` del parser.

Lexer con expresiones regulares.

La codificación manual del autómata en el Micro-Truco facilitó la comprensión del lexer. En la versión final se adoptó el módulo `re` de Python, fundamentándose en que las reglas léxicas son un lenguaje regular y en que las expresiones regulares ofrecen una alternativa más concisa y mantenible.

3. Conclusiones

El proyecto verificó la viabilidad de modelar reglas cotidianas como las del Truco mediante lenguajes formales, consolidando los conceptos centrales de la asignatura y mostrando la aplicabilidad de la teoría de la computación a problemas concretos.

Se cumplieron los seis objetivos específicos. Se definió formalmente una GLC no ambigua, verificada como $LL(1)$ mediante FIRST y FOLLOW [3]. Se implementó un lexer con “*re*” que produce tokens con información de línea y descarta comentarios. Se construyó un parser descendente recursivo que genera un AST y aplica modo pánico para la recuperación de errores. Se diseñaron 55 casos de prueba cuyos resultados confirmaron que el sistema acepta cadenas válidas y rechaza las inválidas, reportando línea y token problemático.

El desarrollo del analizador consolidó los conceptos de gramáticas libres de contexto, autómatas finitos y de pila, análisis descendente y manejo de errores. La lectura del capítulo “2. *A Map Of The Territory*” [1] proporcionó una visión panorámica de las fases de un compilador (desde el scanning hasta la generación de código) que permitió ubicar con precisión el alcance del proyecto dentro del frente de la implementación. Esta perspectiva ayudó a comprender que el parser construido es, en esencia, el mismo primer escalón que emplean compiladores e intérpretes de lenguajes de programación reales. La elección del Truco como dominio de aplicación no fue arbitraria: se buscó un entorno lúdico y familiar que, por su aparente simplicidad, hiciera visibles los fundamentos formales que sostienen la tecnología moderna que, por su omnipresencia, suelen pasar desapercibidos.

La metodología colaborativa basada en roles y el control de versiones con Git posibilitaron una

integración ordenada del código. La bitácora de aprendizaje documentó el proceso y constituye evidencia del trabajo en equipo.

3.1 Posibles extensiones

El sistema construido abarca exclusivamente el frente del proceso de compilación (*scanning* y *parsing*), dejando abierta la posibilidad de completar el resto del recorrido descrito en la metáfora de la montaña de Nystrom [2]. Como línea de trabajo futuro, se proyecta la implementación de un módulo de análisis semántico que recorra el AST generado y valide restricciones lógicas que exceden la capacidad de una gramática libre de contexto, tales como el control de turnos de canto entre los jugadores, la consistencia de los puntos acumulados o la prohibición de cantar envideo después de la primera mano. Este módulo constituiría la cima de la montaña, el punto de máxima comprensión del programa, y sentaría las bases para descender hacia una representación ejecutable.

Un desarrollo aún más ambicioso consistiría en recorrer el resto del descenso: transformar el AST en una representación intermedia y, a partir de ella, generar código para una máquina virtual que simule completamente una partida de Truco. Esta extensión cerraría el círculo entre la especificación formal del lenguaje y su ejecución, y demostraría de manera completa la utilidad de los conceptos de la teoría de la computación en un dominio no convencional.

Bibliografía

- [1] M. Sipser, *Introduction to the Theory of Computation*, 3rd ed. Boston, MA: Cengage Learning, 2013.
- [2] R. Nystrom, *Crafting Interpreters*. Genever Benning, 2021. [En línea]. Disponible en: <https://craftinginterpreters.com/>
- [3] A. V. Aho, M. S. Lam, R. Sethi, and J. D. Ullman, *Compilers: Principles, Techniques, and Tools*, 2nd ed. Boston, MA: Addison-Wesley, 2006.
- [4] Python Software Foundation, “re - Regular expression operations”, Python 3.13 Documentation, 2026. [En línea]. Disponible en: <https://docs.python.org/3/library/re.htm>